Actividad JavaScript Fundamentos

Instrucciones

● Las actividades deben entregarse siempre en formato PDF (en el caso de un solo documento) /ZIP (varios documentos o ejercicios) para evitar desconfiguraciones de formato con el siguiente nombre:
DWEC_Act1_ApellidosNombre.pdf

● Si no se respetan las instrucciones de entrega, se penalizará con 0,25 puntos por instrucción incumplida.

● Si no se entrega a tiempo la actividad, la calificación equivaldrá a 0.

● La entrega de la actividad se debe realizar únicamente a través del campus. La entrega de la tarea por otros medios no tiene validez.

● El trabajo debe ser completamente original. Si existe la sospecha de que no has realizado la tarea, se puede pedir que defiendas tu práctica. Si no puedes justificar tu trabajo o responder las preguntas que se te plantean, la calificación de toda la tarea equivaldrá a 0.

UD2 – JavaScript Fundamentos: Ejercicios evaluables

Desarrollo Web en Entorno Cliente

Instrucciones de entrega

1. Crea un **repositorio público** en GitHub llamado `UD2-JS-Fundamentos-TuNombre-Apellido`.
2. Estructura del repositorio:
   - `README.md` (tu nombre, curso y un índice con enlaces a cada ejercicio).
   - Una carpeta por ejercicio: `ej01/`, `ej02/`, ..., `ej10/`.
   - Dentro de cada carpeta: `index.html` y `script.js` (el JavaScript siempre en archivo externo, enlazado con `<script src="script.js"></script>`).
3. Haz **al menos un commit por ejercicio** con un mensaje descriptivo (por ejemplo: `ej03: añadido conversor de tipos`). Se valorará el historial de commits.
4. Entrega la **URL del repositorio** en la tarea del aula virtual antes de la fecha límite.
5. Si el repositorio es privado, no está accesible o está vacío, la tarea se calificará con 0.

Criterios generales de evaluación

- Cumple lo que pide el enunciado y funciona sin errores en consola.
- Usa `let` y `const` correctamente (nunca `var`) y nombres de variables claros en camelCase.
- Código legible: una instrucción por línea, sangrado correcto y comentarios útiles.
- Estructura de repositorio y commits según las instrucciones.

Cada ejercicio vale 1 punto (total: 10 puntos). Un punto se reparte así: funcionamiento (0,6), calidad y limpieza del código (0,2), comentarios y commit (0,2).

Ejercicio 1 – Hola, JavaScript (script externo y sentencias)

Crea una página HTML que enlace un archivo `script.js` externo. En el script, con modo estricto activado, muestra dos `alert` consecutivos: uno con tu nombre y otro con el nombre del módulo. Añade un comentario de una línea y otro multilínea explicando qué hace el código.

Conceptos: etiqueta `<script>`, `"use strict"`, sentencias, comentarios, `alert`.

Ejercicio 2 – Variables y constantes

Declara variables con tus datos: `userName`, `age`, `city` (una línea por variable). Declara una constante `COLOR_FAVORITO` con un valor hexadecimal y una constante `loadTime` calculada en tiempo de ejecución (por ejemplo con `Date.now()`). Muestra todo con `alert` y comprueba qué ocurre si intentas reasignar una constante (deja el intento comentado y anota el error que aparece en consola).

Conceptos: `let`, `const`, nombres válidos y buenas prácticas de nombrado, constantes en mayúsculas frente a constantes calculadas.

Ejercicio 3 – Tipos de datos y `typeof`

Crea una variable de cada uno de los tipos básicos: `number`, `bigint`, `string`, `boolean`, `null`, `undefined` y `symbol`, además de un `object`. Muestra en pantalla (con `document.write` o `console.log`) el valor y el resultado de `typeof` de cada una, en forma de lista. Al final, escribe un comentario explicando por qué `typeof null` devuelve `"object"`.

Conceptos: ocho tipos de datos, `typeof`, valores especiales.

Ejercicio 4 – Plantillas de texto con backticks

Pide con `prompt` el nombre y la ciudad del usuario. Muestra un mensaje de bienvenida usando **backticks** y `${...}` que incluya además el resultado de una expresión (por ejemplo, el año actual menos el año de nacimiento que también debes pedir). Gestiona el caso en que el usuario pulse *Cancelar* (el `prompt` devuelve `null`).

Conceptos: `prompt`, `null`, strings con backticks, interpolación.

Ejercicio 5 – Conversión de tipos

Pide dos números con `prompt` y muestra su **suma correcta** (como números) y lo que ocurre si no conviertes los valores (concatenación). Después, crea una tabla en HTML (generada desde JS) que muestre el resultado de `Number()`, `String()` y `Boolean()` aplicados a estos valores: `"123"`, `" 45 "`, `"abc"`, `""`, `"0"`, `0`, `null`, `undefined`, `true`.

Conceptos: conversión a string, number y boolean, `NaN`, suma frente a concatenación.

Ejercicio 6 – Operadores aritméticos

Crea una mini calculadora por consola/alert que pida dos números y muestre: suma, resta, multiplicación, división, resto (`%`) y potencia (`**`). Añade una segunda parte con una variable `counter` que uses con `++` y `--` en forma de prefijo y sufijo, y explica en comentarios por qué `alert(counter++)` y `alert(++counter)` muestran valores distintos. Usa asignaciones compuestas (`+=`, `*=`) al menos dos veces.

Conceptos: operadores aritméticos, incremento/decremento, asignaciones compuestas.

Ejercicio 7 – Operadores de comparación: igualdad estricta

Prepara un script que evalúe y muestre el resultado (`true`/`false`) de al menos 12 comparaciones, incluyendo estos casos: `2 > 1`, `"2" > 1`, `"01" == 1`, `0 == false`, `"" == false`, `0 === false`, `null == undefined`, `null === undefined`, `null > 0`, `null >= 0`, `undefined == 0` y `"Glow" > "Glee"`. Para cada una añade un comentario que explique **por qué** da ese resultado.

Conceptos: `==` frente a `===`, comparación de strings, `null` y `undefined`.

Ejercicio 8 – Condicionales: `if`, `else if`, `else` y ternario

Pide una nota numérica (0–10) con `prompt` y muestra: *Suspenso* (menos de 5), *Aprobado* (5 a 6,9), *Notable* (7 a 8,9) o *Sobresaliente* (9 o más) usando `if / else if / else`. Valida que el dato sea un número entre 0 y 10 y, si no lo es, muestra un error. Después, reescribe una de las comprobaciones con el **operador ternario** y deja la versión original comentada.

Conceptos: estructuras condicionales, validación, operador `?`.

Ejercicio 9 – Operadores lógicos y `??`

Crea un formulario de alta simulado con `prompt`: nombre, apodo y edad.

1. Muestra el primer valor disponible entre nombre y apodo usando `||`; si no hay ninguno, muestra *Anónimo*.
2. Usa `&&` para mostrar el mensaje *Acceso permitido* solo si la edad es mayor o igual que 18 y hay un nombre o un apodo.
3. Demuestra con un ejemplo propio la diferencia entre `||` y `??` (por ejemplo, con una altura de `0`) y explícalo en un comentario.
4. Usa `!!` para convertir un valor a booleano.

Conceptos: `&&`, `||`, `!`, `??`, valores *truthy* y *falsy*.

Ejercicio 10 – Bucles y `switch`: mini aplicación final

Crea un programa que muestre un menú con `prompt` y se repita hasta que el usuario elija *Salir*. Opciones, resueltas con `switch`:

- **1** – Mostrar la tabla de multiplicar de un número (con `for`).
- **2** – Sumar números introducidos por el usuario hasta que introduzca uno vacío o cancele (con `while(true)` y `break`).
- **3** – Mostrar solo los números impares del 1 al 20 (con `for` y `continue`).
- **4** – Cuenta atrás desde un número dado hasta 0 (con `do...while`).
- **0** – Salir.

Incluye una opción por defecto (`default`) para valores desconocidos y comentarios que indiquen qué tipo de bucle usas en cada caso.

Conceptos: `while`, `do...while`, `for`, `break`, `continue`, `switch`.

Ejercicio 11 – Gestor de PlayList

Implementa un script que gestione una playlist con un **array bidimensional**: cada fila es una canción y cada columna un dato de esa canción, con el formato **[título, artista, duración en segundos, género]**.

```
const playlist = [
  ["Bohemian Rhapsody", "Queen", 354, "Rock"],
  ["Hotel California", "Eagles", 391, "Rock"],
  ["Imagine", "John Lennon", 183, "Pop"]
];
```

Recuerda que `playlist[1][0]` o `playlist.at(1).at(0)` accede a la fila 1, columna 0 ("Hotel California").

Parte 1: push y pop

1. Añade al final de la playlist dos canciones nuevas (cada una es un array de 4 elementos).
2. Elimina la última canción y guarda en una variable la fila eliminada.
3. Añade un quinto dato (por ejemplo, el año) a la primera canción.
4. Muestra en el documento:
    - El título de la canción eliminada.
    - La playlist resultante con todos sus datos.

Parte 2: splice

5. Inserta una nueva canción en la posición 1 de la playlist sin eliminar ninguna de las canciones existentes.
6. Elimina la fila situada en la posición 2 y guarda la canción eliminada en una variable.
7. Sustituye la canción situada en la posición 0 por una canción diferente.
8. Muestra en el documento:
    - Todos los datos de la canción eliminada.
    - La playlist resultante con todos sus datos.

Parte 3: spread y copias

9. Crea una copia de la playlist utilizando el operador **spread (. . .)**. A continuación, modifica el título de la primera canción en la copia y comprueba qué ocurre en la playlist original. Explica mediante un comentario en el código por qué se produce ese comportamiento.

10. Crea ahora una **copia independiente de los arrays internos** utilizando `map` y spread. Repite la modificación anterior y comprueba que, en este caso, cambiar la copia no modifica la playlist original.

Conceptos: arrays bidimensionales, `push`, `pop`, `splice`, `at`, spread, copia superficial frente a copia independiente, `map`.

Ejercicio 12 – ¿Cuántos días faltan para un evento especial?

Desarrolla una aplicación que determine la fecha de un evento especial que se celebra en un día aleatorio del mes actual.

- Genera un día aleatorio entre 1 y 30 para la fecha del evento. Si el día es anterior al día de la fecha actual, se vuelve a generar.
- Calcula cuántos días faltan desde la fecha actual hasta ese día.
- Muestra el resultado utilizando **Template Strings**.
- Haz un script con el objeto **Date** y otro con la API **Temporal**.

Ejemplo de salida esperada:

```
La fecha actual es 20 de septiembre de 2026
La fecha del evento: 22 de septiembre de 2026
Días que faltan para el evento: 2 días
```

Conceptos: `Date`, API `Temporal`, `Math.random()`, bucles de repetición hasta cumplir una condición, template strings.

Nota: la API Temporal todavía no está disponible en todos los navegadores. Comprobad su compatibilidad en https://caniuse.com y, si hace falta, usad un polyfill.